Studio Nx

Backend(FastAPI/Python) 60 — update_project()

2026-10-08

Backend(FastAPI/Python) 60 — `update_project()` (제목 · 설명 수정)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API — 프로젝트 정보 수정 · PRD 01 FR-124
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	59 `create_project` · 61 `delete_project`

1. 하는 일

Edit details 모달에서 보낸 **제목 · 설명 중 보낸 칸만** 바꾼다. OWNER · EDITOR 만 할 수 있다.

보낸 값 (PATCH 본문)	결과
`{"title": " New Title "}`	제목 `New Title`, 설명 그대로, `updated_at` 갱신
`{"description": "new desc"}`	설명만 바뀜, 제목 그대로
`{"description": null}` · `""` · `" "`	설명 지움 (None)
`{}`	아무것도 바뀌지 않음, `updated_at` 도 그대로
`{"title": "T", "bpm": 200}`	제목만 바뀜 — BPM 은 이 API 로 못 바꿈 (시퀀서에서, Phase 2)
VIEWER 가 보냄	403 `"You don't have permission."`, 아무것도 안 바뀜
`{"title": null}` · `{"title": ""}`	422 — 여기까지 오지 않음 (스키마 56)

2. 언제 쓰나

호출하는 곳	화면
`PATCH /api/v1/projects/{id}` 라우터 (62) — `get_membership`(57) 으로 `project` · `member` 를 받은 뒤	myProject.html 카드의 Edit details 모달 Save (P1-14)

라우터는 돌려받은 프로젝트를 `to_item`(58) 으로 바꿔 응답한다 → 화면은 그 카드 한 장만 다시 그린다.

3. 코드

import (파일 맨 위 — 59 와 같음, 이번에 추가한 것 없음)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
from app.models.track import TrackType
from app.schemas.project import (
```

```
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

이 함수는 `Session` · `Project` · `ProjectMember` · `MemberRole` · `ProjectUpdateRequest` 를 쓴다 (역할 검사는 57 `require_role`).

함수

```
def update_project(
    db: Session,
    project: Project,
    member: ProjectMember,
    data: ProjectUpdateRequest,
) -> Project:
    require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    changes = data.model_dump(exclude_unset=True)
    for name, value in changes.items():
        setattr(project, name, value)
    db.commit()
    db.refresh(project)  # 새 updated_at 읽기
    return project
```

문법 풀이

import 문법	뜻
(새 import 없음)	59 에서 추가한 `MemberRole` 을 역할 검사에 씀

식	뜻
`data.model_dump(exclude_unset=True)`	**본문에 실제로 있던 칸만** 사전으로. `{"title": "A"}` → `{"title": "A"}` (description 없음)
`for name, value in changes.items():`	사전의 (이름, 값) 쌍을 하나씩
`setattr(project, name, value)`	`project.title = value` 와 같음 — 칸 이름이 변수일 때
`require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	역할이 둘 중 하나가 아니면 403 (57)

`exclude_unset` 이 중요한 이유:

보낸 본문	`model_dump()`	`model_dump(exclude_unset=True)`
`{"title": "A"}`	`{"title": "A", "description": None}` → 설명이 **지워짐** ✗	`{"title": "A"}` ✓
`{"description": null}`	`{"title": None, "description": None}`	`{"description": None}` → 설명만 지움 ✓

4. 흐름

1. 역할 검사 — VIEWER 면 403, 아무것도 안 바꿈.

2. 보낸 칸만 꺼낸다.
3. 그 칸만 객체에 넣는다.
4. `commit` — 바뀐 칸만 `UPDATE`. 값이 바뀌었으면 DB 가 `updated_at` 을 지금으로.
5. `refresh` 로 새 `updated_at` 을 읽어 돌려줌.

5. 왜 이렇게

선택	이유
`exclude_unset=True`	보내지 않은 칸까지 None 으로 덮어쓰면 제목만 고쳐도 설명이 사라진다 (PATCH = 일부만 수정)
역할 검사를 **맨 앞**	VIEWER 의 요청은 객체를 건드리기 전에 끝낸다
`setattr` 반복	칸이 늘어도(Phase 2 의 키 · 스케일 등) 스키마에만 추가하면 됨. 스키마에 없는 칸(bpm 등)은 pydantic 이 버려서 들어올 수 없다
`updated_at` 을 직접 넣지 않음	DB 의 `ON UPDATE CURRENT_TIMESTAMP(3)` 가 맡음 — 빈 요청에는 UPDATE 가 없으니 시각도 그대로 (목록의 "최근 수정순"이 정확)
`try/rollback` 없음	한 행 UPDATE 하나라 반쯤 저장될 일이 없다. 실패하면 요청이 끝날 때 세션이 닫히며 취소됨 (59 는 세 행이라 필요)
`title.strip()` · null 검사 없음	스키마(56)가 이미 처리 (템플릿의 "title null → 422" 를 스키마로 옮김)

6. 시험 (`tests/test_project_service.py` · 9개 추가, 모두 38개)

시험	확인한 것
제목만	공백 정리된 제목, 설명 그대로, `updated_at` 이 10월 1일보다 뒤
설명만	제목 그대로
설명 지우기 3가지	null · `""` · `" "` → None
빈 본문	제목 · 설명 · `updated_at` 모두 그대로
스키마에 없는 칸	`bpm: 200` 은 무시 → 120.00
EDITOR	수정 가능
VIEWER	403, DB 의 제목 그대로

전체 시험 476개 통과, `ruff` 검사 통과.